Segment Tree

1 Introduction

A Segment Tree is a binary tree data structure used to efficiently process range queries and updates on an array.

It supports:

- Range Sum Query (RSQ)
- Range Minimum Query (RMQ)
- Range Maximum Query (RMaxQ)

Segment Tree works for both **static and dynamic arrays**.

2 Segment Tree Structure

- Each node represents a range $[l, r]$
- Root represents range $[0, n-1]$
- Left child: $[l, mid]$
- Right child: $[mid+1, r]$

Height: $O(\log n)$ **Number of nodes:** $\leq 4n$

3 Example Array

$$A = [1,\ 3,\ 5,\ 7,\ 9,\ 11,\ 2,\ 6]$$

$$n = 8$$

—

4 Range Sum Query (RSQ)

4.1 Definition

Find the sum of elements in the range $[L, R]$.

4.2 Numerical Question 1

Query: RSQ(0, 3)

$$1 + 3 + 5 + 7 = 16$$

Segment Tree Result: 16

4.3 Numerical Question 2

Query: RSQ(2, 6)

$$5 + 7 + 9 + 11 + 2 = 34$$

Segment Tree Result: 34

—

5 Range Minimum Query (RMQ)

5.1 Definition

Find the minimum element in the range $[L, R]$.

5.2 Numerical Question 1

Query: RMQ(1, 4)

$$\min(3,\ 5,\ 7,\ 9) = 3$$

Segment Tree Result: 3

5.3 Numerical Question 2

Query: RMQ(3, 7)

$$\min(7,\ 9,\ 11,\ 2,\ 6) = 2$$

Segment Tree Result: 2

—

6 Range Maximum Query (RMaxQ)

6.1 Definition

Find the maximum element in the range $[L, R]$.

6.2 Numerical Question 1

Query: RMaxQ(0, 5)

$$\max(1,\ 3,\ 5,\ 7,\ 9,\ 11) = 11$$

Segment Tree Result: 11

6.3 Numerical Question 2

Query: RMaxQ(4, 7)

$$\max(9,\ 11,\ 2,\ 6) = 11$$

Segment Tree Result: 11

—

7 Algorithms

7.1 Build Segment Tree

Algorithm 1 Build Segment Tree

```
1: function BUILD(node, start, end)
2:     if start = end then
3:         tree[node] ← A[start]
4:     else
5:         mid ← (start + end)/2
6:         Build(left child)
7:         Build(right child)
8:         tree[node] ← merge(left, right)
9:     end if
10: end function
```

7.2 Query Segment Tree

Algorithm 2 Segment Tree Query

```
1: function QUERY(node, start, end, L, R)
2:     if R < start or L > end then
3:         return identity value
4:     end if
5:     if L ≤ start and end ≤ R then
6:         return tree[node]
7:     end if
8:     mid ← (start + end)/2
9:     return merge(left query, right query)
10: end function
```

Merge function:

- Sum → addition
- Min → minimum
- Max → maximum

8 Time and Space Complexity

- Build Time: $O(n)$
- Query Time: $O(\log n)$
- Update Time: $O(\log n)$
- Space Complexity: $O(n)$

9 Segment Tree vs Sparse Table

Feature	Segment Tree	Sparse Table
Updates	Yes	No
Query Time	$O(\log n)$	$O(1)$
Memory	$O(n)$	$O(n \log n)$

10 Conclusion

Segment Trees provide a flexible and efficient way to handle range queries and updates. They are widely used in competitive programming and real-time systems.